Preprocessing Strategy for Multi-Protocol Dataset Collection

ICMP + TCP + UDP Covert Channel Detection (FYP)

🎯 YOUR CURRENT SITUATION

What You Have:

- ✅ ICMP flow-level dataset: 649 flows (Normal + Flow_Bending)
- ⏳ TCP covert dataset: In progress
- ⏳ UDP covert dataset: In progress
- ⏳ Normal TCP/UDP traffic: In progress

Your Supervisor's Requirements:

1. Collect TCP covert with 3 techniques (Header, Timing, Flow Bending)
2. Collect UDP covert with 3 techniques (Header, Timing, Flow Bending)
3. Collect normal TCP/UDP traffic
4. Combine all protocols into balanced dataset
5. Train multi-protocol Transformer model

Your Question:

- Should you do preprocessing/augmentation NOW or LATER?
- When to split train/val/test?

✅ RECOMMENDED STRATEGY: WAIT & COMBINE

Answer: DO NOT preprocess individual protocol datasets yet!

Rationale:

1. ❌ Don't augment ICMP alone - will create artificial imbalance when combined
2. ❌ Don't split now - may split related flows across train/test
3. ❌ Don't normalize yet - TCP/UDP will have different feature scales
4. ✅ Wait until ALL protocols collected → then preprocess combined dataset

---

📋 PHASED PREPROCESSING PLAN

PHASE 1: DATA COLLECTION (Current - Do This Now)

Focus on collecting consistent flow-level features across all protocols.

1.1 Feature Consistency Requirements

CRITICAL: Ensure EXACT same features for all protocols!

Current ICMP Features (35 columns):

```
flow_id, flow_duration, avg_inter_arrival_time, std_inter_arrival_time,
min_inter_arrival_time, max_inter_arrival_time, timing_entropy,
timing_skewness, timing_kurtosis, burst_ratio, delay_spike_ratio,
total_packets, total_bytes, avg_packet_size, std_packet_size,
min_packet_size, max_packet_size, packet_size_entropy,
packet_size_skewness, non_standard_size_ratio, oversized_packet_ratio,
icmp_type_entropy, icmp_code_entropy, icmp_checksum_entropy,
hop_limit_mean, hop_limit_std, dscp_entropy, ecn_entropy,
flow_label_entropy, unique_src_ips, unique_dst_ips,
bidirectional_flag, flow_packets_per_second, flow_bytes_per_second,
label
```

TCP Features Should Include:

```
# Same base features
flow_id, flow_duration, avg_inter_arrival_time, std_inter_arrival_time,
... (all timing features same as ICMP)

# TCP-specific header features (replace ICMP fields)
tcp_flags_entropy,          # Instead of icmp_type_entropy
tcp_seq_entropy,            # Instead of icmp_code_entropy
tcp_ack_entropy,            # Instead of icmp_checksum_entropy
tcp_window_size_mean,       # Instead of hop_limit_mean
tcp_window_size_std,        # Instead of hop_limit_std

# Keep same flow-level features
dscp_entropy, ecn_entropy, flow_label_entropy,
unique_src_ips, unique_dst_ips, bidirectional_flag,
flow_packets_per_second, flow_bytes_per_second

# Add TCP-specific features
tcp_syn_ratio, tcp_fin_ratio, tcp_rst_ratio,
tcp_urg_count, tcp_psh_count,

label
```

UDP Features Should Include:

```
# Same base features (timing, size, flow-level)
... (all timing features same as ICMP)

# UDP-specific header features (replace ICMP fields)
udp_port_entropy,          # Instead of icmp_type_entropy
udp_length_variance,       # Instead of icmp_code_entropy
udp_checksum_entropy,      # Instead of icmp_checksum_entropy
hop_limit_mean,            # Keep (same as ICMP)
hop_limit_std,             # Keep (same as ICMP)

# Keep same flow-level features
dscp_entropy, ecn_entropy, flow_label_entropy,
unique_src_ips, unique_dst_ips, bidirectional_flag,
flow_packets_per_second, flow_bytes_per_second

label
```

1.2 Label Standardization

CRITICAL: Use consistent multi-class labels across protocols!

Proposed Label Schema:

```python
# Multi-class labels (7 classes)
labels = {
    0: 'Normal',                    # Normal traffic (all protocols)
    1: 'ICMP_Flow_Bending',         # ICMP flow bending only
    2: 'TCP_Timing_Obfuscation',    # TCP timing covert
    3: 'TCP_Header_Manipulation',   # TCP header covert
    4: 'TCP_Flow_Bending',          # TCP flow bending
    5: 'UDP_Timing_Obfuscation',    # UDP timing covert
    6: 'UDP_Header_Manipulation',   # UDP header covert
    7: 'UDP_Flow_Bending',          # UDP flow bending
}
```

OR Hierarchical Schema (recommended for balanced dataset):

```python
```

```
# Protocol + Technique (12 classes)
labels = {
    0: 'Normal_ICMP',
    1: 'Normal_TCP',
    2: 'Normal_UDP',
    3: 'ICMP_Header_Manipulation',
    4: 'ICMP_Timing_Obfuscation',
    5: 'ICMP_Flow_Bending',
    6: 'TCP_Header_Manipulation',
    7: 'TCP_Timing_Obfuscation',
    8: 'TCP_Flow_Bending',
    9: 'UDP_Header_Manipulation',
    10: 'UDP_Timing_Obfuscation',
    11: 'UDP_Flow_Bending',
}
```

1.3 Collection Targets

Minimum Samples per Class (for balanced dataset):

Class	Target Samples	Notes
Normal_ICMP	500-1,000	Currently: 349 ✅
Normal_TCP	500-1,000	⏳ Collect
Normal_UDP	500-1,000	⏳ Collect
ICMP_Timing	200-500	May need to split from current
ICMP_Header	200-500	May need to split from current
ICMP_Flow_Bending	200-500	Currently: 300 ✅
TCP_Timing	200-500	⏳ Collect
TCP_Header	200-500	⏳ Collect
TCP_Flow_Bending	200-500	⏳ Collect
UDP_Timing	200-500	⏳ Collect
UDP_Header	200-500	⏳ Collect
UDP_Flow_Bending	200-500	⏳ Collect

Total Target: 3,000-6,000 flows minimum

1.4 What to Do NOW with ICMP Dataset

DO NOT:

- ❌ Augment data
- ❌ Remove features
- ❌ Normalize/scale
- ❌ Split train/val/test
- ❌ Encode labels

DO:

- ✅ Keep raw ICMP dataset as-is
- ✅ Document exact feature extraction pipeline
- ✅ Create feature extraction script that works for TCP/UDP
- ✅ Verify all features are correctly calculated
- ✅ Store metadata (timestamp, source, technique used)

Save ICMP dataset like this:

```python
# Save with metadata
icmp_df.to_csv('icmp_flows_raw.csv', index=False)

# Save metadata
metadata = {
    'protocol': 'ICMP',
    'collection_date': '2024-XX-XX',
    'total_flows': len(icmp_df),
    'features': list(icmp_df.columns),
    'label_distribution': icmp_df['label'].value_counts().to_dict(),
    'notes': 'Flow_Bending is synthetic. Need to relabel if techniques identified.'
}

import json
with open('icmp_metadata.json', 'w') as f:
    json.dump(metadata, f, indent=2)
```

PHASE 2: PROTOCOL ALIGNMENT (After Collecting TCP/UDP)

Once you have TCP and UDP datasets, align features before combining.

2.1 Feature Mapping

Create unified feature set:

```python
```

```python
import pandas as pd

# Define common feature names (protocol-agnostic)
common_features = [
    'flow_id', 'protocol',  # Add protocol identifier
    'flow_duration', 'avg_inter_arrival_time', 'std_inter_arrival_time',
    'min_inter_arrival_time', 'max_inter_arrival_time',
    'timing_entropy', 'timing_skewness', 'timing_kurtosis',
    'burst_ratio', 'delay_spike_ratio',
    'total_packets', 'total_bytes',
    'avg_packet_size', 'std_packet_size',
    'min_packet_size', 'max_packet_size',
    'packet_size_entropy', 'packet_size_skewness',
    'non_standard_size_ratio', 'oversized_packet_ratio',
    'dscp_entropy', 'ecn_entropy', 'flow_label_entropy',
    'unique_src_ips', 'unique_dst_ips', 'bidirectional_flag',
    'flow_packets_per_second', 'flow_bytes_per_second',
]

# Protocol-specific header features (to be renamed)
protocol_specific_mapping = {
    'ICMP': {
        'icmp_type_entropy': 'header_field_1_entropy',
        'icmp_code_entropy': 'header_field_2_entropy',
        'icmp_checksum_entropy': 'header_field_3_entropy',
        'hop_limit_mean': 'header_field_4_mean',
        'hop_limit_std': 'header_field_4_std',
    },
    'TCP': {
        'tcp_flags_entropy': 'header_field_1_entropy',
        'tcp_seq_entropy': 'header_field_2_entropy',
        'tcp_ack_entropy': 'header_field_3_entropy',
        'tcp_window_size_mean': 'header_field_4_mean',
        'tcp_window_size_std': 'header_field_4_std',
    },
    'UDP': {
        'udp_port_entropy': 'header_field_1_entropy',
        'udp_length_variance': 'header_field_2_entropy',
        'udp_checksum_entropy': 'header_field_3_entropy',
        'hop_limit_mean': 'header_field_4_mean',
        'hop_limit_std': 'header_field_4_std',
    }
}

def align_protocol_features(df, protocol):
    """Align protocol-specific features to common names"""
```

```python
    df_aligned = df.copy()
    df_aligned['protocol'] = protocol

    # Rename protocol-specific features
    rename_map = protocol_specific_mapping[protocol]
    df_aligned = df_aligned.rename(columns=rename_map)

    return df_aligned

# Apply to each protocol
icmp_aligned = align_protocol_features(icmp_df, 'ICMP')
tcp_aligned = align_protocol_features(tcp_df, 'TCP')
udp_aligned = align_protocol_features(udp_df, 'UDP')
```

2.2 Feature Validation

Check feature consistency:

```python
def validate_feature_alignment(dfs, protocol_names):
    """Ensure all dataframes have same features"""

    # Get feature sets
    feature_sets = [set(df.columns) for df in dfs]

    # Find common and unique features
    common = set.intersection(*feature_sets)

    print(f"Common features: {len(common)}")

    for i, (df, protocol) in enumerate(zip(dfs, protocol_names)):
        unique = set(df.columns) - common
        if unique:
            print(f"\n{protocol} unique features: {unique}")
            print(f"  ⚠️  WARNING: {protocol} has {len(unique)} unique features!")
        else:
            print(f"\n{protocol}: ✅ All features aligned")

    return common

# Validate
common_features = validate_feature_alignment(
    [icmp_aligned, tcp_aligned, udp_aligned],
    ['ICMP', 'TCP', 'UDP']
)
```

PHASE 3: DATASET COMBINATION (After Alignment)

Combine all protocols into single dataset.

3.1 Concatenation

```python
import pandas as pd

# Combine all protocols
combined_df = pd.concat([
    icmp_aligned,
    tcp_aligned,
    udp_aligned
], ignore_index=True)

print(f"Combined dataset shape: {combined_df.shape}")
print(f"\nProtocol distribution:")
print(combined_df['protocol'].value_counts())
print(f"\nLabel distribution:")
print(combined_df['label'].value_counts())
```

3.2 Balance Check

```python
```

```python
def check_balance(df):
    """Check class balance across protocols and techniques"""

    print("=" * 80)
    print("CLASS BALANCE ANALYSIS")
    print("=" * 80)

    # Overall balance
    print("\nOverall label distribution:")
    label_counts = df['label'].value_counts()
    for label, count in label_counts.items():
        pct = count / len(df) * 100
        print(f"  {label}: {count} ({pct:.2f}%)")

    # Calculate imbalance ratio
    max_count = label_counts.max()
    min_count = label_counts.min()
    imbalance = max_count / min_count

    print(f"\nImbalance ratio: {imbalance:.2f}:1")

    if imbalance < 2:
        print("  ✅ Well-balanced!")
    elif imbalance < 3:
        print("  ⚠️ Slight imbalance - use class weights")
    else:
        print("  ❌ Significant imbalance - need resampling!")

    # Per-protocol balance
    print("\n" + "=" * 80)
    print("PER-PROTOCOL DISTRIBUTION")
    print("=" * 80)

    for protocol in df['protocol'].unique():
        protocol_df = df[df['protocol'] == protocol]
        print(f"\n{protocol}:")
        print(protocol_df['label'].value_counts())

check_balance(combined_df)
```

3.3 Balancing Strategies

If imbalanced after collection:

```python
```

```python
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.combine import SMOTEENN

def balance_dataset(df, strategy='hybrid'):
    """
    Balance dataset using various strategies

    Parameters:
    - df: Combined dataframe
    - strategy: 'oversample', 'undersample', 'hybrid', 'class_weights'
    """

    X = df.drop(columns=['flow_id', 'label'])
    y = df['label']

    if strategy == 'oversample':
        # SMOTE oversampling
        smote = SMOTE(random_state=42)
        X_balanced, y_balanced = smote.fit_resample(X, y)
        print(f"Original: {len(df)} → Oversampled: {len(X_balanced)}")

    elif strategy == 'undersample':
        # Random undersampling
        rus = RandomUnderSampler(random_state=42)
        X_balanced, y_balanced = rus.fit_resample(X, y)
        print(f"Original: {len(df)} → Undersampled: {len(X_balanced)}")

    elif strategy == 'hybrid':
        # SMOTEENN (SMOTE + Edited Nearest Neighbors)
        smoteenn = SMOTEENN(random_state=42)
        X_balanced, y_balanced = smoteenn.fit_resample(X, y)
        print(f"Original: {len(df)} → Hybrid balanced: {len(X_balanced)}")

    elif strategy == 'class_weights':
        # Don't resample, just compute class weights
        from sklearn.utils.class_weight import compute_class_weight

        classes = y.unique()
        weights = compute_class_weight('balanced', classes=classes, y=y)
        class_weights = dict(zip(classes, weights))

        print("Class weights computed:")
        for cls, weight in class_weights.items():
            print(f"  {cls}: {weight:.4f}")
```

```
        return df, class_weights

    # Recreate dataframe with balanced data
    df_balanced = pd.DataFrame(X_balanced, columns=X.columns)
    df_balanced['label'] = y_balanced

    return df_balanced, None

# Choose strategy based on imbalance
balanced_df, class_weights = balance_dataset(combined_df, strategy='hybrid')
```

PHASE 4: PREPROCESSING (After Balancing)

Now apply preprocessing to final combined dataset.

4.1 Feature Cleaning

```python
```

```python
def clean_features(df):
    """Remove problematic features"""

    # 1. Remove constant features (variance < threshold)
    print("Checking for constant features...")

    numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
    constant_features = []

    for col in numeric_cols:
        if col not in ['flow_id', 'label']:
            if df[col].var() < 1e-10 or df[col].nunique() <= 1:
                constant_features.append(col)
                print(f"  ⚠️  {col}: variance={df[col].var():.10f}, unique={df[col].nunique()}")

    if constant_features:
        print(f"\nRemoving {len(constant_features)} constant features:")
        for feat in constant_features:
            print(f"  - {feat}")
        df_cleaned = df.drop(columns=constant_features)
    else:
        print("  ✅ No constant features found")
        df_cleaned = df.copy()

    # 2. Remove highly correlated features (optional)
    print("\nChecking for highly correlated features...")

    corr_matrix = df_cleaned[numeric_cols].corr().abs()
    upper_triangle = corr_matrix.where(
        np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
    )

    high_corr_features = [
        column for column in upper_triangle.columns
        if any(upper_triangle[column] > 0.95)
    ]

    if high_corr_features:
        print(f"  ⚠️  {len(high_corr_features)} highly correlated features:")
        for feat in high_corr_features[:10]:  # Show first 10
            print(f"  - {feat}")
        print("\n  Consider removing for dimensionality reduction")
    else:
        print("  ✅ No highly correlated features (>0.95)")

    return df_cleaned
```

```
cleaned_df = clean_features(balanced_df)
```

4.2 Feature Scaling

```python
```

```python
from sklearn.preprocessing import StandardScaler, RobustScaler

def scale_features(df, method='standard'):
    """
    Scale features for neural network training

    Parameters:
    - df: Cleaned dataframe
    - method: 'standard', 'robust', 'minmax'
    """

    # Separate features and labels
    feature_cols = [col for col in df.columns
                    if col not in ['flow_id', 'label', 'protocol']]

    X = df[feature_cols].values

    # Choose scaler
    if method == 'standard':
        scaler = StandardScaler()
        print("Using StandardScaler (mean=0, std=1)")
    elif method == 'robust':
        scaler = RobustScaler()
        print("Using RobustScaler (robust to outliers)")
    elif method == 'minmax':
        from sklearn.preprocessing import MinMaxScaler
        scaler = MinMaxScaler()
        print("Using MinMaxScaler (range [0,1])")

    # Fit and transform
    X_scaled = scaler.fit_transform(X)

    # Create scaled dataframe
    df_scaled = df.copy()
    df_scaled[feature_cols] = X_scaled

    # Save scaler for later use (test set, production)
    import joblib
    joblib.dump(scaler, 'scaler.pkl')
    print("✅ Scaler saved to 'scaler.pkl'")

    return df_scaled, scaler

# Scale features
scaled_df, scaler = scale_features(cleaned_df, method='standard')
```

4.3 Label Encoding

```python
from sklearn.preprocessing import LabelEncoder

def encode_labels(df):
    """Encode string labels to integers"""

    le = LabelEncoder()
    df_encoded = df.copy()

    # Encode labels
    df_encoded['label_encoded'] = le.fit_transform(df['label'])

    # Save label encoder
    import joblib
    joblib.dump(le, 'label_encoder.pkl')

    # Print mapping
    print("Label encoding:")
    for i, label in enumerate(le.classes_):
        count = (df_encoded['label_encoded'] == i).sum()
        print(f"  {i}: {label} ({count} samples)")

    return df_encoded, le

encoded_df, label_encoder = encode_labels(scaled_df)
```

PHASE 5: TRAIN/VAL/TEST SPLIT (Final Step)

Only split AFTER all preprocessing!

```python
```

```python
from sklearn.model_selection import train_test_split

def split_dataset(df, test_size=0.2, val_size=0.1):
    """
    Split dataset with stratification

    Parameters:
    - df: Preprocessed dataframe
    - test_size: Test set proportion (default 20%)
    - val_size: Validation set proportion (default 10%)
    """

    # Separate features and labels
    X = df.drop(columns=['flow_id', 'label', 'label_encoded', 'protocol'])
    y = df['label_encoded']

    # Keep protocol for analysis
    protocol = df['protocol']

    # First split: train+val vs test
    X_temp, X_test, y_temp, y_test, proto_temp, proto_test = train_test_split(
        X, y, protocol,
        test_size=test_size,
        random_state=42,
        stratify=y  # Ensure balanced split
    )

    # Second split: train vs val
    val_proportion = val_size / (1 - test_size)
    X_train, X_val, y_train, y_val, proto_train, proto_val = train_test_split(
        X_temp, y_temp, proto_temp,
        test_size=val_proportion,
        random_state=42,
        stratify=y_temp
    )

    # Print split statistics
    print("=" * 80)
    print("DATASET SPLIT SUMMARY")
    print("=" * 80)

    print(f"\nTrain set: {len(X_train)} samples ({len(X_train)/len(df)*100:.1f}%)")
    print(f"Val set: {len(X_val)} samples ({len(X_val)/len(df)*100:.1f}%)")
    print(f"Test set: {len(X_test)} samples ({len(X_test)/len(df)*100:.1f}%)")

    # Check balance in each split
```

```
    for split_name, split_y in [('Train', y_train), ('Val', y_val), ('Test', y_test)]:
        print(f"\n{split_name} label distribution:")
        for label in sorted(split_y.unique()):
            count = (split_y == label).sum()
            pct = count / len(split_y) * 100
            print(f"  Class {label}: {count} ({pct:.1f}%)")

    # Check protocol distribution
    print("\n" + "=" * 80)
    print("PROTOCOL DISTRIBUTION PER SPLIT")
    print("=" * 80)

    for split_name, split_proto in [('Train', proto_train), ('Val', proto_val), ('Test', proto_test)]:
        print(f"\n{split_name}:")
        print(split_proto.value_counts())

    return (X_train, X_val, X_test), (y_train, y_val, y_test)

# Split dataset
(X_train, X_val, X_test), (y_train, y_val, y_test) = split_dataset(encoded_df)

# Save splits
X_train.to_csv('X_train.csv', index=False)
X_val.to_csv('X_val.csv', index=False)
X_test.to_csv('X_test.csv', index=False)
pd.DataFrame(y_train).to_csv('y_train.csv', index=False)
pd.DataFrame(y_val).to_csv('y_val.csv', index=False)
pd.DataFrame(y_test).to_csv('y_test.csv', index=False)

print("\n✅ Splits saved to CSV files")
```

PHASE 6: DATA AUGMENTATION (If Needed)

Only augment training set, not val/test!

```
python
```

```python
def augment_training_data(X_train, y_train, augmentation_factor=2):
    """
    Augment training data with noise injection

    Parameters:
    - X_train: Training features
    - y_train: Training labels
    - augmentation_factor: How many augmented copies (2 = 3x total size)
    """

    print(f"Original training size: {len(X_train)}")

    X_augmented = [X_train]
    y_augmented = [y_train]

    for i in range(augmentation_factor):
        # Add Gaussian noise (5% of feature std)
        noise = np.random.normal(0, 0.05, X_train.shape)
        X_noisy = X_train + noise

        X_augmented.append(X_noisy)
        y_augmented.append(y_train)

    # Combine
    X_train_aug = pd.concat(X_augmented, ignore_index=True)
    y_train_aug = pd.concat(y_augmented, ignore_index=True)

    print(f"Augmented training size: {len(X_train_aug)} ({augmentation_factor+1}x)")

    return X_train_aug, y_train_aug

# Augment if training set < 2000 samples
if len(X_train) < 2000:
    factor = max(1, 2000 // len(X_train))
    X_train_aug, y_train_aug = augment_training_data(
        X_train, y_train,
        augmentation_factor=factor
    )
else:
    print("Training set large enough, no augmentation needed")
    X_train_aug, y_train_aug = X_train, y_train
```

📋 YOUR IMMEDIATE ACTION PLAN

WEEK 1-2: Focus on Collection

Tasks:

1. ✅ Keep ICMP dataset as-is (raw, no preprocessing)
2. 🎯 Create consistent feature extraction script for TCP
3. 🎯 Create consistent feature extraction script for UDP
4. 🎯 Collect TCP covert data:
   - Timing obfuscation: 200-500 flows
   - Header manipulation: 200-500 flows
   - Flow bending: 200-500 flows
5. 🎯 Collect UDP covert data:
   - Timing obfuscation: 200-500 flows
   - Header manipulation: 200-500 flows
   - Flow bending: 200-500 flows
6. 🎯 Collect normal TCP traffic: 500-1,000 flows
7. 🎯 Collect normal UDP traffic: 500-1,000 flows

Target: 3,000-6,000 total flows

WEEK 3: Feature Alignment

Tasks:

1. Align protocol-specific features to common schema
2. Validate feature consistency across protocols
3. Check for missing features
4. Document feature mappings

WEEK 4: Dataset Combination

Tasks:

1. Combine all protocols into single dataset
2. Check class balance
3. Apply balancing strategy if needed (SMOTE/undersampling)
4. Validate final dataset

WEEK 5: Preprocessing

Tasks:

1. Remove constant features
2. Scale features (StandardScaler)
3. Encode labels
4. Split train/val/test (70/15/15)
5. Augment training set if < 2,000 samples

WEEK 6+: Model Training

Start Transformer training!

⚠️ CRITICAL WARNINGS

DON'T DO NOW:

1. ❌ **DON'T augment ICMP dataset alone**
   - Will create imbalance when combined
   - Augment after combining all protocols
2. ❌ **DON'T split ICMP dataset**
   - May split related flows
   - Split after combining all protocols
3. ❌ **DON'T normalize/scale ICMP alone**
   - TCP/UDP have different scales
   - Normalize after combining
4. ❌ **DON'T remove features from ICMP**
   - Need consistency with TCP/UDP
   - Remove after feature alignment
5. ❌ **DON'T start training yet**
   - Need complete multi-protocol dataset
   - Wait for full collection

DO NOW:

1. ✅ **Keep ICMP raw dataset intact**
2. ✅ **Document feature extraction process**
3. ✅ **Create reusable extraction scripts**

4. ✅ **Focus on collecting TCP/UDP data**
5. ✅ **Ensure label consistency**

📊 CHECKLIST: Are You Ready to Combine?

Use this checklist when you've collected all protocols:

- [ ] All protocols collected (ICMP, TCP, UDP)
- [ ] All techniques present (Timing, Header, Flow Bending)
- [ ] Normal traffic collected for all protocols
- [ ] Same feature count across all protocols
- [ ] Features have consistent names/meanings
- [ ] Labels are standardized
- [ ] Minimum 200 samples per class
- [ ] Total samples > 2,000
- [ ] Metadata documented for all datasets

If all checked → Proceed to Phase 2 (Alignment)

💡 FINAL SUMMARY

Your Current Task: COLLECT, DON'T PROCESS

Next Steps:

1. Collect TCP covert (3 techniques) + normal
2. Collect UDP covert (3 techniques) + normal
3. Come back with all datasets
4. Then follow Phases 2-6 above

ETA to Start Training: 4-6 weeks (including collection + preprocessing)

When You're Ready: Return with all protocols collected, then we'll help with alignment and preprocessing!

Good luck with data collection! 🚀